

Plano de retomada — Agentes & Tarefas do BioByte

Data de início: 11/09/2026

Projeto de demonstração: BioByte Sentinela

Etapas: Agentes & Tarefas (Agent-Task Spec)

Ponto de partida do repositório: commit 309c31f

1. Objetivo

Regenerar e validar, pela interface do LangNet, o documento de **Agentes & Tarefas** a partir da Especificação Funcional e do Modelo de Dados atualmente aprovados do BioByte.

O resultado não será apenas uma lista de agentes. Ele deverá ser um contrato executável para as etapas seguintes: Ferramentas, YAML, Fluxo/Petri, Casos de Teste, Código e Deploy.

2. Entendimento da arquitetura que será respeitado

O aplicativo gerado não executa tarefas como uma lista linear simples. A Rede de Petri é o orquestrador:

```
place (lugar) → transição → place → transição → ...
    |
    └─ lógica JavaScript monta entradas, chama uma task Python via WebSocket,
        recebe a saída e entrega o resultado ao próximo place
```

Cada tarefa precisa, portanto, declarar claramente:

- qual agente a executa;
- quais entradas recebe da tela, do banco ou de places anteriores;
- qual regra de negócio deve ser aplicada;
- quais ferramentas externas utiliza;
- quais campos devolve;
- quais tarefas dependem de sua saída;
- em que lugar e transição ela poderá ser executada;
- como sua execução será rastreada até o UC e os requisitos funcionais.

Uma tarefa que parece correta isoladamente, mas não recebe os valores do place anterior ou não devolve os campos esperados pelo próximo, não está correta.

3. Fontes que serão usadas

As fontes serão selecionadas explicitamente na interface, com versão visível:

1. Especificação Funcional mais recente e aprovada do BioByte.
2. Modelo de Dados correspondente àquela especificação.

3. UI Spec e Protótipo já regenerados e validados.
4. Documentos complementares do BioByte, quando a interface permitir sua seleção.
5. Catálogo de ferramentas MCP global e ferramentas habilitadas no projeto.

Não será usado um artefato antigo apenas porque está disponível no banco. A origem e a versão serão conferidas antes da geração.

4. Resultado esperado

O documento deve conter, para cada agente e tarefa:

- nome estável e compatível com `agents.yaml` e `tasks.yaml`;
- papel do agente e limites de responsabilidade;
- casos de uso e requisitos cobertos;
- entradas e saídas estruturadas;
- dependências entre tarefas;
- tarefas determinísticas, de agente e externas classificadas corretamente;
- regras de negócio preservadas, sem transformar uma decisão em texto genérico;
- ferramentas MCP nomeadas pelo identificador real;
- persistência e tabelas coerentes com o Modelo de Dados;
- mensagens de sucesso, recusa e falha externa vindas do caso de uso;
- matriz de cobertura e rastreabilidade.

Para o BioByte, conferirei especialmente login/MFA, importação microbiológica, classificação NHSN, detecção de MDR, escore de Cox, recomendação de tratamento, dashboard, auditoria, relatórios e o canal de e-mail MCP.

5. Plano de execução

Fase 0 — Preparação e proteção do ponto de partida

1. Confirmar que o commit `309c31f` está no remoto.
2. Confirmar que não existem alterações do usuário misturadas às alterações da retomada.
3. Subir o backend e a interface no ambiente `langnet`, sem iniciar uma segunda cópia.
4. Conferir saúde da API, autenticação, projeto BioByte e disponibilidade das sessões.
5. Conferir o provedor configurado. O padrão atual é DeepSeek; Qwen local e Claude Code devem continuar disponíveis como alternativas comentadas, sem apagar configuração.
6. Registrar o saldo/custo do DeepSeek antes de qualquer chamada que consuma o modelo.

Saída: ambiente identificado, serviços respondendo e ponto de partida preservado.

Fase 1 — Seleção e conferência das fontes

1. Abrir a etapa Agentes & Tarefas pela rota do projeto.
2. Escolher a Especificação Funcional pelo nome, data e versão, não somente pelo UUID.
3. Conferir se a especificação contém os 14 casos de uso esperados e os campos clínicos relevantes, incluindo o escore de Cox e o APACHE II.
4. Conferir a correspondência com o Modelo de Dados e a UI Spec.
5. Fotografar a tela de origem escolhida e registrar IDs/versões no relatório.

Saída: conjunto de fontes definido e auditável.

Fase 2 — Geração do documento pela interface

1. Solicitar a geração usando o botão da etapa e o Playwright.
2. Acompanhar o job por polling até conclusão ou falha explícita.
3. Validar que o documento não está vazio, truncado ou reduzido a um resumo superficial.
4. Conferir a quantidade de agentes, tarefas, casos de uso e requisitos cobertos.
5. Conferir se as tarefas mantêm nomes que possam virar chaves válidas de YAML/Python.
6. Salvar captura da etapa pronta e o documento produzido como evidência.

Saída: primeira versão do Agent-Task Spec persistida no LangNet.

Fase 3 — Revisão estrutural e de negócio

A revisão será feita no documento e pela conversa da própria etapa. Não serão feitas correções manuais diretamente no banco ou no artefato gerado.

Conferências obrigatórias:

- nenhum UC relevante ficou sem agente ou tarefa;
- nenhuma tarefa concentra sozinho requisitos demais sem justificativa;
- tarefas de CRUD não substituem tarefas de decisão clínica;
- cálculo de Cox não é tratado como simples CRUD;
- consulta microbiológica e escore de Cox usam as ferramentas MCP reais;
- alertas MDR preservam a condição de multirresistência e o envio de e-mail;
- entradas vindas de tarefas anteriores têm nomes e tipos definidos;
- cada saída necessária ao place seguinte está declarada;
- exceções externas possuem mensagem funcional compreensível;
- dados sensíveis, como senha e MFA, não são devolvidos nem exibidos;
- persistência usa tabelas e colunas presentes no Modelo de Dados.

As correções serão enviadas em instruções pequenas e verificáveis, por exemplo: “detalhar a entrada que vem do place de autenticação e a saída consumida pelo place de importação”. Após cada refino, será conferida a mudança efetiva e o que permaneceu inalterado.

Saída: versão refinada, com decisões de negócio explícitas.

Fase 4 — Conferência de compatibilidade com Petri e WebSocket

Antes de aprovar, o documento será comparado com o modelo de execução:

1. Cada tarefa será relacionada a um possível place.
2. As dependências serão verificadas contra uma ordem topológica possível.
3. Entradas e saídas serão comparadas com a lógica JavaScript dos places.
4. Será conferido se uma saída é serializável e transportável pelo WebSocket.
5. Serão identificados loops, dependências inexistentes, saídas nulas e referências a tarefas inexistentes.
6. Tarefas paralelizáveis serão separadas das que dependem de resultado anterior.
7. Ferramentas externas serão marcadas para o caminho correto: chamada determinística pelo programa ou julgamento do agente com dados previamente buscados.

Saída: especificação pronta para alimentar YAML e Petri sem perda de contrato.

Fase 5 — Aprovação e passagem controlada

1. Aprovar a versão pela interface.
2. Registrar versão, origem, revisão e evidências.
3. Só depois iniciar a etapa Ferramentas.
4. Não regenerar YAML, Petri ou código antes da aprovação desta etapa.
5. Se houver falha, preservar a versão anterior e criar uma nova versão, sem sobrescrever o entregável existente.

Saída: Agent-Task Spec aprovado e apto a ser consumido pela próxima etapa.

6. Validações específicas do BioByte

Área	O que será conferido
Autenticação	usuário, senha, MFA/TOTP e saída sem segredo
Microbiologia	chamada à ferramenta MCP e persistência das amostras
NHSN	dados necessários para classificação e resultado explícito
MDR	condição de multirresistência, alerta e destinatário
Cox	entradas clínicas, chamada MCP e campos do score
Tratamento	ligação entre classificação, risco e bundle recomendado
Dashboard	consultas, agregações e métricas declaradas
Relatório	dados de origem, formato PDF e persistência
E-mail	ferramenta <code>enviar_email</code> , falha externa e mensagem funcional
Petri	encadeamento, paralelismo e nomes de places/transições

7. Critérios de aceite

A etapa será considerada concluída somente quando:

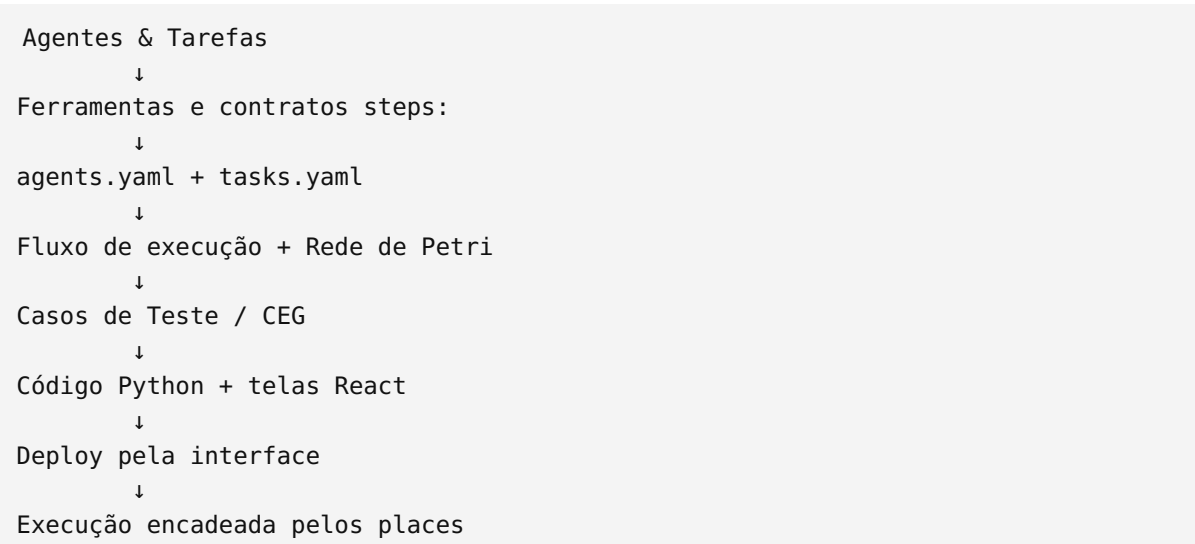
- houver uma versão aprovada pela interface;
- todos os UCs tiverem cobertura identificável;
- todas as tarefas tiverem agente, entradas, saídas e dependências;
- tarefas MCP estiverem ligadas às ferramentas reais ou marcadas explicitamente como pendentes;
- não houver tarefa de cálculo relevante descrita apenas em prosa;
- o documento for compatível com a sequência de places e transições;
- a matriz de rastreabilidade estiver preenchida;
- o artefato estiver pronto para Ferramentas/YAML;
- o custo do DeepSeek e a evidência da execução estiverem registrados.

8. Riscos e respostas

Risco	Resposta
Especificação errada selecionada	escolher por nome, data e versão e fotografar a origem
Documento resumido pelo modelo	comparar cobertura e refinar por UC, sem aceitar resumo genérico
Timeout do provedor	usar geração em fases, polling e limite dinâmico de tokens
MCP omitido do agente	conferir catálogo, atribuição por agente e saída estruturada
Tarefa incompatível com Petri	validar entradas/saídas contra places antes de aprovar
YAML inválido	exigir nomes sanitizados e validar antes da etapa seguinte
Perda de entregável	criar versão nova; nunca sobrescrever a versão aprovada
Custo inesperado	registrar saldo antes/depois e aproveitar cache de contexto

9. Sequência posterior

Depois da aprovação desta etapa, a retomada seguirá estritamente:





O teste final não será uma chamada isolada de task. Será a execução encadeada pela Rede de Petri, com os outputs de um place alimentando o seguinte e com os casos de sucesso e falha externa verificados no aplicativo implantado.

10. Entregáveis desta retomada

- este plano em Markdown e PDF;
- capturas da seleção da origem, geração, revisão e aprovação;
- versão aprovada do Agent-Task Spec;
- relatório de cobertura UC/FR/agente/task;
- registro do custo do modelo;
- lista de pendências que devem ser resolvidas nas etapas Ferramentas e YAML.